Labo 4: Geavanceerde Computerarchitectuur

Gust Brouckaert
Tim Depickere

15 november 2024

1 Introductie

In dit vierde labo kijken we eerst naar de prestatie winst door het gebruik van coalesced memory access tegenover coalesced memory access. Dit weer door de rood waarde in een pixel te inverteren voor een foto. In het 2de deel van dit labo wordt er gekeken naar het verschil in prestatie bij het gebruik van verschillende memory types. Hier zullen 2 matrixes vermenigvuldigd worden, dit wordt gedaan met gebruik van Global, shared, en constant memory.

2 Coalesced memory access

Bij coalesced memory access wordt bereikt als alle threads in een warp access hebben tot opeenvolgende globale geheugenlocaties locaties. De hardware combineert al deze accesses tot een geconsolideerde access tot opeenvolgende DRAM-locaties

Om coalesced memory access te hebben bij een foto waarvan enkel de rood waarden van de pixel geïnverteerd moeten worden zullen we deze foto opslaan als een 1D array waar eerst alle rood waarden komen dan alle groen waarden en tenslotte alle blauw waarden **(RRR...GGG...BBB)**.

Bij de uncoalesced memory access wordt de indexering van de RGB waarden anders gedaan dan bij de coalesced memory access, voor de rest zijn deze functies gelijk

```
// Kernel voor uncoalesced memory access (RGBRGBRGB format)
__global__ void invertRedUncoalesced(uint8_t* image, uint8_t* output) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int deel = M * N;
    for (int i = idx; i < deel; i +=stride){
        if (idx < deel) {
            int base_idx = idx * 3;
            output[base_idx] = 255 - image[base_idx];
            output[base_idx + 1] = image[base_idx + 1];
            output[base_idx + 2] = image[base_idx + 2];
        }
    }
}
```

```
// Kernel voor coalesced memory access (RRRR...GGGG...BBBB format)
__global__ void invertRedCoalesced(uint8_t* image, uint8_t* output) {
    int deel = M * N;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    for (int i = idx; i < deel; i +=stride){
        if (idx < deel) {
            output[idx] = 255 - image[idx];
            output[idx + deel] = image[idx + deel];
            output[idx + 2 * deel] = image[idx + 2 * deel];
        }
    }
}
```

We zien een duidelijk verschil tussen de 2 kernels, coalesced memory access presteert zoals verwacht beter dan de uncoalesced memory access dit is te zien in foto 1.

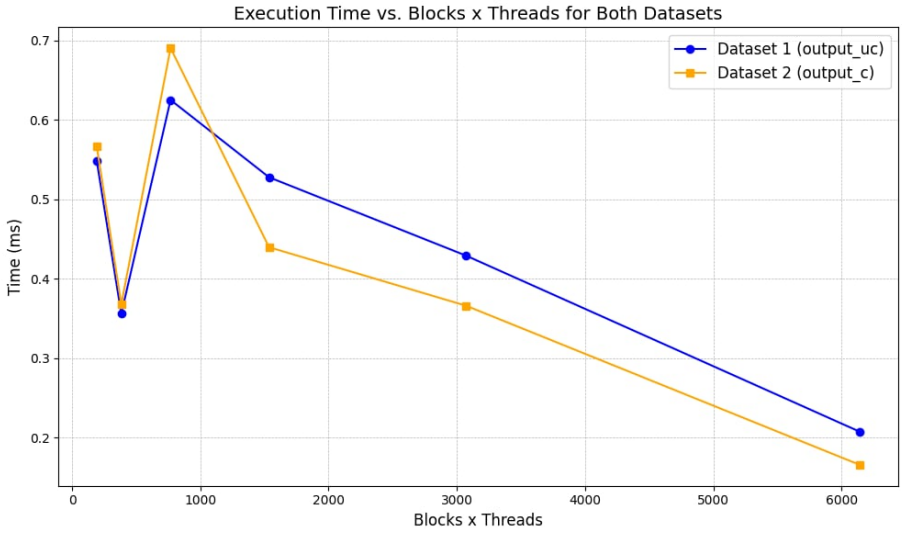


Figure 1: Runtimes: coalesced memory access vs. uncoalesced memory access

3 Global, shared, and constant memory

Voor het gebruik van Global memory moesten we niets speciaal doen aangezien dit het standaard memory type is bij gebruik van cudaMemcpy.

Voor constant memory defineerden we eerst de matrix:

```
__constant__ int d_B_const[MATRIX_SIZE * MATRIX_SIZE];
```

Om deze matrix in het constant memory, read only, te krijgen gebruikte we de functie cudaMemcpyToSymbol. Aangezien we slechts 64Kb aan constant memory hebben moest er wel rekening met de grootte van de matrix gehouden worden.

Bij shared memory werdt de shared memory pas in de kernel zelf aangemaakt:

```
__shared__ int s_A[BLOCK_SIZE][BLOCK_SIZE];
__shared__ int s_B[BLOCK_SIZE][BLOCK_SIZE];
```

Aangezien we slecht 48Kb aan shared memory hebben maakten we gebruik van kleinere tiles die we elk apart oplosten.

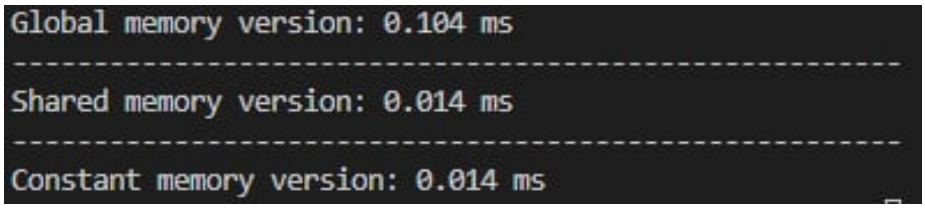


Figure 2: Runtimes: Global memory vs Constant memory vs Shared Memory

In figuur 2 ziet u de prestaties van de verschillende memory types. Zoals verwacht is Shared memory de snelste, wel valt het op dat constant memory even snel is dit komt door het gebruik van tiles bij shared memory. Hierdoor zal het iets trager gaan aangezien constant memory dit niet moet doen. Constant memory is sneller dan global memory, de grootste reden hiervoor is dat het read-only memory is en er dus nooit synchronisatie moet gebeuren. Echter is deze memory wel beperkter dan global memory.